Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Tasarım kalıbı nedir? (ekrandan, 00:20:00)
  - Tasarım kalıbı, yazılım tasarımında karşılaşılan belirli sorunların yalın ve güzel çözümlerinin tarifidir. Çözüm somut bir gerçekleme olmak zorunda değildir, soyut bir tarif de olabilir.
  - Amaç: tekerleği yeniden keşfetmeden "iyi" tasarıma ulaşmak. İyi tasarım, geçen dersteki tanımla problemin parçalanabildiği düzeyde parçalanan tasarımdır; temel ölçütleri uyum ve bağlaşımdır. Sorunların esnek biçimde ve yeniden kullanılabilirliği artıracak yönde çözülmesi beklenir.
  - Esneklik: gereksinim değişiklikleri kaçınılmazdır; tasarım ve kod bu değişikliklere kolay uyum sağlayabilmelidir.
- Bir tasarım kalıbının dört ana bileşeni (ekrandan, 00:20:00)
  - İsim: kalıbı en iyi anlatan, kolay hatırlanan bir ya da birkaç kelimelik isim.
  - Sorun: kalıbın çözdüğü sorunun tarifi.
  - Çözüm: önerilen tasarımın bileşenleri (sınıflar), bunların ilişkileri, sorumlulukları ve birlikte çalışmaları.
  - Tartışma/Sonuçlar: çözümün esneklik, genişletilebilirlik, taşınabilirlik gibi yönlerden sisteme etkileri; güçlü ve zayıf yönleri, yer-zaman çelişkileri, başka kalıplarla işbirliği olanakları.
- Kalıplar arasındaki olası ilişkiler (ekrandan, 00:23:00)
  - GoF kitabındaki ilişki şemasında örneğin Command, Composite ile birleştirilebilir ("composed using"); Strategy ve State nesneleri Flyweight ile paylaşılabilir; Abstract Factory tek örnekli olarak Singleton ile, dinamik yapılandırma için Prototype ile, gerçekleme için Factory Method ile ilişkilidir.
  - Şema bütün olası ilişkileri göstermez. Hoca örnek olarak strateji nesnelerinin Singleton olabileceğini, ama bu ilişkinin şemada çizilmediğini gösterdi. Yeni problemlerde yeni etkileşimler keşfedilebilir.
- Tasarım kalıplarının yararları (ekrandan, 00:30:00)
  - Tekerleği yeniden icat etmemek.
  - Üzerinde birçok deneyimli kişinin emeği geçtiğinden "kaliteli" bir çözüm.
  - Deneyimin yazılı biçimde aktarılabilmesi. Deneyim normalde usta-çırak ya da öğretmen-öğrenci ilişkisinde gösterilerek aktarılır; kalıplar ayrıntılı şablonlar olduğu için deneyimi yazılı aktarır.
  - Yazılım ekibi arasında ortak bir sözlük oluşturması. Kalıp adlarını bilen iki tasarımcı iki cümlede tasarımın önemli bir kısmına karar verebilir.
  - Tartışma: bir kişinin "kalıp" dediğine diğeri "basit bir yapı taşı" ya da "temel bir ilke" diyebilir. Hocaya göre basit de olsa yararlı ve kullanışlı olduğu sürece kalıp adını hak eder.
- Kalıpların sınıflandırılması ve anlatım sırası (ekrandan, 00:40:00)
  - Amaca göre üç grup: yaratımsal (creational), yapısal (structural), davranışsal (behavioral). GoF kitabı ve ders bu sırayla ilerler; ilk anlatılan kalıp ise MVC'dir.
  - Yaratımsal kalıplar nesnelerin oluşturulması ve ilişkilendirilmesindeki bağımlılığı (bağlaşımı) azaltmaya yöneliktir.
  - Yapısal kalıplar sınıf ve nesneleri bir araya getirerek daha büyük yapılar elde etmeye ve bu ilişkilerin idamesine yöneliktir.
  - Davranışsal kalıplar sorumlulukların nesnelere dağıtılmasına (yüksek uyum) ve

doğru algoritma seçimine yöneliktir.
  - Odağa göre iki grup: sınıfsal kalıplar sınıflar ve alt sınıfları arasındaki ilişkilere yöneliktir; bu ilişkiler derleme zamanında belirlenir ve durağandır. Nesnesel kalıplar nesneler arası ilişkilere yöneliktir; bu ilişkiler çalışma süresince rahatlıkla değiştirilebilir. Adapter hem sınıfsal hem nesnesel sayılır, diğer kalıplar yalnız birine girer.
- Model-View-Controller (MVC) kalıbı (ekrandan, 00:53:40, 01:00:00, 01:30:00)
  - Sorun: aynı veriyi farklı biçimlerde göstermek; kullanıcıya verinin nasıl ve ne kadarının gösterileceğini seçme, veriyi değiştirme olanağı vermek.
  - Çözüm: "Neyin değiştiğini bul ve onu sarmala" ilkesiyle veri (Model), gösterim (View) ve gösterimin ve verinin yönetimi (Controller) ayrı sınıflara konur.
  - İlişkiler: Controller → View, Controller → Model, View → Model. Model hiçbir sınıfa bağımlı değildir.
  - Yararlar: gösterim, kontrol ve veri ayrıldığı için bunlar çalışma anında değiştirilebilir, ilgi alanları ayrılır, yeniden kullanım artar.
  - Çeşitlemeler: Model'in değişiklikleri View'a bildirmesi; birden fazla View (iç içe, ayrı ya da tek görevcikte, örtüşen ya da ayrık); birden fazla Controller (klavye, fare, ses gibi farklı denetim biçimleri).
- Abstract Factory kalıbı (ekrandan, 01:31:10, 01:40:00, 01:50:00, 01:51:50)
  - Amaç: birbiriyle ilişkili ya da aynı aileden olan nesneleri, sınıflarını belirtmeden oluşturabilmek.
  - Örnek: Java'da GUI bileşenlerinin görünüşünün (look-and-feel) çalışma anında değiştirilmesi; farklı Linux dağıtımlarında GNOME, KDE gibi pencere yöneticilerinden herhangi biriyle çalışabilmek.
  - Sorun: taşınabilirlik için istekçi, hangi tür görünüşün hangi sınıftan nesnelerle gerçeklendiğini bilmemelidir.
  - Kalıp yapısı: soyut AbstractFactory sınıfında her ürün için bir CreateProductX() soyut metodu bulunur. Her ürün ailesi için bir ConcreteFactory alt sınıfı yazılır. Ürünler soyut AbstractProductA, AbstractProductB sınıflarından türeyen somut sınıflardır. Client yalnız soyut fabrika ve soyut ürünleri tanır.
  - Uygulanabileceği anlar: sistem ürünlerinin nasıl oluşturulduğundan, birleştirildiğinden ve sunulduğundan bağımsız olmalıysa; sistem farklı ailelerden herhangi birindeki nesnelerle yapılandırılacaksa; ilişkili ürün ailelerinin nesneleri birlikte kullanılacaksa; bir ürün ailesi sınıf kütüphanesi olarak sunulacak ve yalnız arayüzleri açığa çıkarılacaksa.
- Örnek uygulama: 3B oyun motoru (ekrandan, 01:56:00, 02:00:00, 02:10:00, 02:21:00)
  - Problem: işletim sistemi ve ekran kartına göre OpenGL ya da DirectX kütüphanesinin renderer, shader gibi bileşenleri kullanılacak.
  - Çözüm 1 (switch-case): LibraryType enum'u (OpenGL, DirectX), her kütüphane ve bileşen için ayrı üye alan; doRendering ve doShading metotlarında kütüphane türüne göre switch-case.
  - Çözüm 2 (kalıtım ve çok biçimlilik): soyut Renderer ve Shader sınıfları, bunlardan türeyen OpenGL ve DirectX sınıfları. Switch-case yalnız ApControl kurucusunda kalır.
  - Çözüm 3 (Abstract Factory): soyut ComponentFactory (createRenderer, createShader) ve somut OpenGLFactory, DirectXFactory. ApControl kurucuya bir ComponentFactory nesnesi alır; ana sınıfta hiç switch-case kalmaz.
- Çıkartılan dersler (tasarıma ait genel kurallar) (ekrandan, 02:30:00)
  - Neyin değiştiğini bul ve onu sarmala: değişen şey soyutlanır, kara kutu yaklaşımıyla kullanılır, değişiklikler yalnız o kısmı etkiler. Örnekte hangi grafik kütüphanesinin kullanılacağı ComponentFactory sınıfında soyutlandı.
  - Kalıtım ilişkisi yerine parça/bütün ilişkisini tercih et.
  - Gerçeklemelere göre değil, arayüzlere göre tasarla: ApControl, ComponentFactory'den bir renderer ve bir shader ister; nasıl oluşturulduğu umurunda değildir.

Hocanın Özellikle Vurguladığı Kısımlar

- Tasarım kalıbı sihirli değnek (silver bullet) değildir
  - Mühendis, çözüm yollarının hangi problemde verimli olduğunu ve getireceği dezavantajları hesaplayan kişidir. Kalıpların güçlü ve zayıf yönleri, yer-zaman çelişkileri (time-memory trade-off) her zaman tartılmalıdır.
  - Kalıplar modellemeye hitap eder, çalışma zamanı başarımına değil. Bazı yerlerde

başarıma olumsuz etkisi olabilir, ama bir işlemin 100 ms yerine 200 ms sürmesi, sağlanan esnekliğin yanında kayıp sayılmaz.
- Kalıplara harfiyen uymak gerekmez
  - Kalıp bir yasa değildir; temel önerilerinde ihtiyaca göre değişiklik yapılabilir. MVC'de Model'den View'a bildirim oku eklemek bunun örneğidir.
- Switch-case görürseniz tasarımı gözden geçirin
  - Kod yazarken kendinizi switch-case yazarken görüyorsanız, büyük olasılıkla orada kalıtım hiyerarşisi ve çok biçimlilik kurmanız ya da sorumlulukları doğru atamanız gerekir.
  - Switch-case ile if-else if zinciri aynı mantıktır, ikisi de aynı soruna yol açar.
- GoF şemaları UML standardına uymaz, siz UML'ye uygun çizin
  - GoF kitabı UML standartlaşmadan önce yazıldığı için şemalarında soyut sınıf ve metotlar yalnız italik yazılır, kalıtım üçgeni üst sınıftan uzakta çizilir (veritabanındaki karga ayağı gösterimine benzer).
  - UML'de soyut sınıfın adının üstüne «abstract» stereotipi, soyut metodun yanına {abstract} yazılır; kalıtım okunun ucu üst sınıfa değecek biçimde çizilir. Metodun dönüş türü UML'de parantezden sonra iki nokta ile yazılır.
  - Hoca, öğrencilerin kendi şemalarını mutlaka güncel UML kurallarına göre çizmesini özellikle istedi; kalıtım okunun yanlış çizilmesi UML sınıf şeması kurallarını bilmemenin en bariz göstergesidir.
- "Factory" tek başına bir kalıp adı değildir
  - Adında Factory geçen kalıp nesne oluşturmayla ilgilidir, ama GoF'ta Abstract Factory ve Factory Method olmak üzere iki ayrı kalıp vardır. Hangisi kastedildiği açıkça söylenmelidir.
- Kalıtım en durağan ilişki türüdür
  - Bir nesnenin sınıfı çalışma anında değiştirilemez: Çalışan sınıfından bir nesne, Öğrenci'ye dönüştürülmek istenirse ancak kopya kurucu ile yeni nesne oluşturulur ve nesnenin kimliği kaybolur. Sahiplik ilişkisiyle (rol üye alanı) aynı iş çalışma anında yapılabilir.
- Çözüm 2'den Çözüm 3'e geçiş yumuşaktır
  - Hocaya göre bu, Çözüm 2'nin nesneye yönelimin ilkelerine gerçekten uyan bir çözüm olduğunun göstergesidir. Çözüm 1 kesinlikle kullanılmamalı; 2 ya da 3 projeye göre seçilebilir.

Kısa Tekrar Notları

- Tasarım kalıbı = sık görülen bir tasarım sorununun yalın, güzel, yeniden kullanılabilir çözümünün tarifi.
- Dört bileşen: isim, sorun, çözüm, tartışma/sonuçlar.
- Yararlar: tekerleği yeniden icat etmemek, kaliteli çözüm, deneyimin yazılı aktarımı, ortak sözlük.
- Gruplar: yaratımsal (bağlaşımı azaltır), yapısal (büyük yapılar kurar), davranışsal (sorumluluk dağıtımı ve algoritma seçimi). Sınıfsal (durağan) ve nesnesel (dinamik) ayrımı; Adapter ikisinde de var.
- MVC: Model hiçbir şeye bağımlı değil; View → Model; Controller → View ve Model. Benzetme: Model = resmi yapılan kişi, View = resmin bakış açısı ve çerçevesi, Controller = ressam.
- MVC çeşitlemeleri: Model → View bildirimi (yoklama yerine), birden fazla View ve Controller (alt sınıflar ile).
- Abstract Factory: ilişkili nesne ailelerini somut sınıf adı vermeden oluşturur. Yeni aile = yeni somut fabrika + yeni somut ürün sınıfları; mevcut koda dokunulmaz.
- Oyun motoru: Çözüm 1 (switch-case) yeni kütüphanede her yeri değiştirir; Çözüm 2 (kalıtım) switch'i kurucuya indirir; Çözüm 3 (Abstract Factory) switch'i ana sınıftan tamamen çıkarır, karşılığında 3 ek sınıf getirir.
- Genel kurallar: değişeni bul ve sarmala; kalıtım yerine parça/bütün; arayüze göre tasarla.

Detaylı Açıklamalar

- MVC'nin çalışması ve çeşitlemeleri

  - Model ham veriyi temsil eder ve nasıl gösterildiğinden ya da nasıl değiştirildiğinden habersizdir. Bu, nesneye yönelimdeki temel bir olguya dayanır: bir nesnenin metodu çağrıldığında, nesne kendisini kimin çağırdığını bilmez.
  - View hangi veriyi göstereceğini bilmek zorunda olduğu için Model'e sahiplik ilişkisiyle bağlıdır. Controller hem veriyi düzenler hem de View ayarlarını (hangi bölümün gösterileceği gibi) değiştirir.
  - Örnek (ekrandan, 01:02:30): Excel'de A, B, C partileri ve Diğer için 38, 27, 15, 20 oy oranı (toplam 100). Aynı veri tablo, pasta grafiği ya da sütun grafiği olarak gösterilebilir. Veri değişse de (örneğin demografik bilgiler) grafik bileşenleri, yani View'lar, başka uygulamada yeniden kullanılabilir.
  - Seçim gecesi örneğinde veri sandıklar açıldıkça başka kaynaklardan değişir. View belli aralıklarla Model'i yoklarsa ya bayat veri gösterme riski ya da sık yoklamanın ağ trafiği ve haberleşme maliyeti vardır. En hızlı güncelleme isteniyorsa Model → View bildirim ilişkisi eklenir; bu da bağlaşımı artırır. Karar bağlaşım ile başarım arasında, hangisi önemliyse ona göre verilir.
  - İç içe View: Türkiye haritasında il düzeyinde oranlar, bir ile odaklanınca seçim bölgesi düzeyinde oranlar. Örtüşen View: bir şirketin hissesi hem BIST 100 hem Teknoloji 50 endeksinde yer alabilir. Farklı görevciklerde çalışan ve veriyi değiştirebilen View'larda veri tutarlılığı ayrıca sağlanmalıdır.
  - Birden fazla View ya da Controller varsa kalıtımla kardeş sınıflar kurulur: View'dan PieChart ve BarChart; varsayılan klavye-fare kontrolcüsünden ses komutlarını da işleyen bir alt sınıf. Çalışma anında setView'a PieChart ya da BarChart nesnesi verilerek gösterim değiştirilir.
- Abstract Factory'nin işleyişi (ekrandan, 01:40:00)
  - GoF örneğinde Client, soyut WidgetFactory'den CreateScrollBar() ve CreateWindow() ile bileşen ister. MotifWidgetFactory ve PMWidgetFactory alt sınıfları sırasıyla MotifScrollBar/MotifWindow ve PMScrollBar/PMWindow döndürür. Güncel karşılıkları GNOME ve KDE fabrikaları olabilir.
  - Client bütün ilişkileri soyut üst sınıflar türünden kurduğu için, kendisine MotifWidgetFactory verildiğinde aslında MotifScrollBar alır, ama bunu bilmez; yalnız soyut ScrollBar'ın metotlarını çağırır. Bu, kalıtımda yerine geçebilirlik ilkesinin kullanımıdır.
  - Yeni bir pencere yöneticisi çıktığında yeni bir kardeş fabrika sınıfı ve o yöneticinin bileşenlerini modelleyen yeni sınıflar yazmak yeterlidir. Müşteriye yalnız yeni sınıflar gönderilir.
  - Alternatif switch-case çözümünde bir enum ile aile seçilir ve her create metodunda bütün aileler için case yazılır. Yeni aile geldiğinde enum'a değer eklenir, bütün switch'ler değiştirilir, mevcut fabrikanın kaynak kodu değişir ve bütün sınıflar yeniden dağıtılmak zorunda kalır.
- Oyun motoru örneğinin üç çözümü (ekrandan, 02:00:00, 02:08:00, 02:10:00, 02:20:00, 02:21:00)
  - Çözüm 1'de yeni bir kütüphane (örneğin DirectX 11) eklemek için yeni Renderer ve Shader sınıfları, enum'a yeni değer, ApControl'e yeni üye alanlar ve doRendering ile doShading'deki her switch'e yeni case gerekir. Texture mapping, ray tracing, çarpışma algılama gibi her yeni işlem bu switch'leri çoğaltır.
  - Çözüm 2'de soyut Renderer ve Shader sınıfları soyut işlemleri (renderOpA/B/C, shadeOpA/B/C) tanımlar, OpenGL ve DirectX alt sınıfları bunları gerçekler. Switch-case yalnız kurucuda, uygun nesneleri oluşturmak için kalır. Yeni kütüphane "tak-çalıştır" biçiminde eklenir.
  - Çözüm 3'te ComponentFactory soyut sınıfı createRenderer() ve createShader() metotlarını tanımlar. OpenGLFactory ve DirectXFactory kendi ailelerinin nesnelerini döndürür. ApControl kurucuya ComponentFactory alır ve r ile s'yi fabrikadan oluşturur.
  - Karşılaştırma: Çözüm 3, Çözüm 2'ye göre 3 ek sınıf getirir ve yeni kütüphane için bir fabrika sınıfı fazladan yazılır. Kazanç, switch-case'in iş mantığına odaklanan sınıflardan tamamen uzaklaştırılmasıdır. Hangi fabrikanın verileceği kararı bir yerde (yapılandırma ön yüzünde) yine switch-case gerektirir.
  - Ek sınıfların yönetimine değip değmeyeceği projenin boyuna ve vakaya göre kararlaştırılır. Yeni kütüphane kesinlikle eklenmeyecekse Çözüm 1 ile devam edilebilir; esnekliğe ihtiyaç doğuran şey tam da yeni kütüphanelerin eklenmesidir.

- Arayüz kavramı
  - Derste “arayüz” bir nesnenin public metotlarının imzalarının tümü anlamında kullanılır, Java interface kavramı kastedilmez. Soyut sınıfların üye alanı ve somut metodu yoksa Java’da interface olarak da tanımlanabilirler; ama hoca dersi dilden bağımsız tutmak için bunu tercih etmedi.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.